WebAssembly GCを活用したCommon Lispコンパイラ実装の技術仕様

序論

WebAssembly（Wasm）は、ブラウザおよびサーバー環境におけるポータブルなバイナリコードフォーマットとして、計算集約型のタスクに革命をもたらしてきた。初期のWasm（Wasm 1.0）は、C、C++、Rustのような手動メモリ管理を前提とする静的型付け言語向けに設計されており、メモリモデルは単一の線形メモリ（Linear Memory）に限定されていた[1]。この制約下で動的型付け言語やガベージコレクション（GC）を必須とする言語（SchemeやCommon Lispなど）を実装する場合、コンパイラは線形メモリ内に独自のGC機構を構築して同梱する必要があり、結果としてバイナリサイズの肥大化や、ホスト環境（JavaScriptエンジンなど）のGCとの相互運用性の欠如という深刻な問題を抱えていた[1]。

この状況を根本から覆すのが、Wasm GC提案（WasmGC）の導入である。WasmGCは、WebAssemblyのコア仕様を拡張し、仮想マシン（VM）ネイティブなヒープ割り当て可能な構造体（struct）および配列（array）を提供する[7]。これにより、言語処理系は独自のGCを線形メモリに実装することなく、V8などのホストVMが提供する高度に最適化されたGCにメモリ管理を委譲することが可能となった[7]。

本仕様書は、WasmGCをターゲットとしてCommon Lispコンパイラをゼロから設計・実装する開発者に向けた包括的な技術仕様である。Common Lispの複雑なデータ構造（シンボル、文字列、コンスセル、クロージャなど）とWasmGCの型階層とのマッピング手法、動的スコープや多様なラムダリスト（&optional、&rest、&key）を処理するための呼出規約、そして例外処理提案（Wasm EH）を用いた大域脱出の実装アーキテクチャについて、多角的な洞察を提供する。

WebAssembly GCの基礎アーキテクチャと型階層

Common Lispの動的型付けシステムをWasm上で表現するためには、WasmGCが提供する参照型の階層構造を深く理解し、それらを効果的に利用する必要がある。WasmGCは、従来の整数や浮動小数点数といったスカラー値に加えて、ホスト管理のヒープオブジェクトを指す参照型（Reference Types）を導入している。これらの参照型は厳密な階層構造（サブタイピング）を持っており、最上位クラスとしてanyref（すべてのWasmGCオブジェクトのルート）が存在する[8]。

anyrefの下には、同値比較が可能なeqrefがあり、さらにその下位に具体的なデータ構造を表現するstructref（構造体）、arrayref（配列）、およびi31ref（31ビットスカラー整数）が定義されている[10]。

Common Lispは動的型付け言語であるため、コンパイル時に変数の型を完全に決定することは困難である。したがって、WasmGCバックエンドにおいて変数が保持しうる汎用的なLisp値（ユニタイプ: Unitype）の表現として、コンパイラは主に(ref null eq)または(ref null any)をベース型として採用することになる[10]。Guile HootなどのSchemeコンパイラの実装においても、変数のデフォルト型として(ref eq)が利用されており、具体的な型は実行時に検査（ダウンキャスト）されるアーキテクチャが標準となっている[12]。

データの型判定とダウンキャストには、WasmGCのref.testおよびref.cast命令を使用する。ref.cast

は、対象の参照が指定された型（例えば特定の構造体）に一致するかを検証し、一致すれば静的に型付けされた参照をスタックに積むが、不一致の場合はトラップ（実行時エラー）を発生させる[15]。コンパイラは、Lispの型述語（conspやsymbolpなど）を評価する際、暗黙のビット演算によるポインタのタグチェックを行うのではなく、Wasmエンジンのランタイム型情報（RTT）を活用したref.test命令をエミットする形へパラダイムを移行させなければならない。この型検査の仕組みは、C言語などで実装された従来のLisp処理系がポインタの下位ビットをマスクして型を判定していたアプローチからの決定的な脱却を意味する。

Common Lispデータ構造のWasmGCマッピング設計

Lispの各データ構造をWasmGCのプリミティブにどのようにマッピングするかは、コンパイラのパフォーマンスとメモリ効率を決定づける最重要の設計課題である。静的型付けの世界であるWasm上で動的型付けのオブジェクトシステムを構築するには、各データ型に最適なヒープ割り当て戦略を選択する必要がある。

即値型 (Immediate Types) とボクシングの回避

ヒープ割り当てのオーバーヘッドを回避するため、小さな整数などの値はポインタ自体に値を埋め込む「即値（Immediate）」として表現することが伝統的なLisp処理系の定石である。WasmGCでは、この目的のためにi31refという特殊な型が提供されている[12]。i31refは、最下位ビットをタグビットとして予約し、残りの31ビットに整数値を格納する参照型である[16]。これにより、メモリ割り当てを行うことなく、anyref互換の参照スタック上で整数を扱うことができる。

Common Lispのfixnumは、このi31refに直接マッピングされる。ただし、Wasmのi31refは符号ビットを含めて31ビットであるため、表現可能な整数の範囲は -2^{30} から $2^{30}-1$ に厳密に制限される[14]。これを超える多倍長整数（bignum）については、後述する構造体（struct）を用いた明示的なボクシング（ヒープ割り当て）が必要となる。

文字（Character）も同様に即値として表現可能である。Unicodeのコードポイントは最大21ビットを必要とするため、31ビットの空間に十分に収まる。コンパイラはi31refの上位ビットに型タグ（例えば、Fixnumなら0、Characterなら1）を付与し、残りのビットにデータを含めることで、単一のi31ref空間を複数の即値型で多重化する設計を採用すべきである。このタグ付け戦略により、GCはこれらの値を追跡すべきポインタではなく、スカラー値として扱うことが可能となり、スキャンのオーバーヘッドが削減される[16]。

Boolean型に関しては、Common Lisp固有の深い課題が存在する。Schemeにおいては、偽値#fと空リスト'()は明確に区別されるため、これらをi31refの特殊なタグ付き値として表現することが可能である[14]。しかし、Common LispではNILは偽値であり、空リストであり、さらにシンボルでもあるという多重の性質を持つ。したがって、NILを単なるヌル参照（ref.null any）や即値として表現することは不可能に近い。なぜなら、(symbol-name nil)や(symbol-plist nil)といったシンボルとしての操作が許容されているためである。この制約から、NILは専用の静的グローバル構造体（Symbol構造体）としてメモリ上に単一インスタンスとして割り当てられなければならない。

コンスセル (Cons Cell) とリスト表現

コンスセルは、carとcdrの2つの要素を持つ極めてシンプルなヒープオブジェクトである。WasmGCにおけるコンスセルの定義は、2つのミュータブルな参照フィールドを持つ構造体として定義される。

Bob Schemeの実装例においても、ペア（PAIR）は同様に2つの(mut (ref null eq))を持つ構造体として定義されている[17]。

フィールド名	**WasmGC** 型宣言	機能的役割
car	(mut (ref null any))	リストの要素、またはドット対の左側。動的型変更（rplaca）を許容するためmut宣言が必須。
cdr	(mut (ref null any))	次のコンスセルへの参照、またはドット対の右側。同様にrplacdのためにミュータブルとする。

構造体の各フィールドへのアクセスは、WasmGCのstruct.getおよびstruct.set命令を通じて行われる[11]。これらの命令はVMレベルでインライン化と最適化の対象となるため、線形メモリに対するオフセット計算を伴う従来のメモリアクセスと比較して、高い局所性と実行速度が期待できる[7]。さらに、WasmGCの構造体はホストのGCによって管理されるため、Lisp側でフリーリストを管理したり、マーク・アンド・スイープのトラバーサルコードを生成したりする必要がなくなる。

文字列 **(String)** とベクタ **(Vector)** の動的配列化

文字列とベクタは、可変長の連続したデータ列であるため、WasmGCの配列（Array）プリミティブを活用する。WasmGCの配列は、生成時に長さが決定されるオブジェクトであり、array.getやarray.setを用いて要素にアクセスする[12]。配列外アクセスが発生した場合はVMが自動的にトラップを発生させるため、言語処理系側で境界チェック（Bounds Checking）の命令を明示的に挿入する必要がなくなり、コンパイル後のコードサイズが削減される[8]。
文字列の場合、UTF-8表現として(array (mut i8))を採用するか、UTF-16として(array (mut i16))を採用するかの選択肢がある。WasmGCは1バイトや2バイトのパックされた要素型（Packed storage type）をサポートしており、アクセス時には自動的な符号拡張（Sign extension）を伴う[18]。汎用的なCommon Lispのベクタ（例えば、任意のLispオブジェクトを格納するsimple-vector）は、(array (mut (ref null any)))として定義される。配列の動的な拡張（adjust-array）が要求される場合、背後にあるWasmGC配列を再割り当てし、要素をコピーするラッパー構造体（フィルポインタと実体配列への参照を持つ構造体）を設計する必要がある。

シンボル **(Symbol)** とパッケージ **(Package)** の構造

Common Lispにおけるシンボルは、単純な識別子ではなく、言語の中核を成す複雑なデータ構造である。各シンボルは、名前（Name）、パッケージ（Package）、値（Value）、関数（Function）、および属性リスト（Plist）という5つの基本スロットを保持している[20]。これらをWasmGCで表現するための構造体設計は極めて重要であり、以下のように静的な型付けが行われる。

スロット名	**WasmGC** 型宣言	機能と役割
name	(ref $string_array)	シンボルの文字列表現。イミュータブルな参照として保持される。
package	(ref null $package_struct)	このシンボルがインターンされているホームパッケージへの参照[20]。
value	(mut (ref null any))	グローバルまたはスペシャル変数としての現在の動的バインディング値[20]。
function	(mut (ref null $closure_struct))	このシンボルに束縛されているグローバル関数またはマクロ[20]。
plist	(mut (ref null any))	プロパティリスト（通常はコンスセルによるリスト表現）[20]。

シンボルは通常、パッケージ内に「インターン（Intern）」され、同一の名前を持つシンボルは同一のメモリアドレス（参照）を持つことが保証される[20]。パッケージ自体は、シンボル名（文字列）からシンボル構造体（オブジェクト）へのマッピングを保持するコンテナとして機能する[20]。Wasm上でインターン機構を実装するには、パッケージごとにシンボルテーブルを構築する必要がある。WasmGCの配列と線形メモリをハイブリッドに活用し、ハッシュテーブルのバケット配列をWasmGCの(array (ref null $symbol))で管理しつつ、キーとなる文字列のハッシュ値計算は高速化のためにWasmの算術命令で処理するアーキテクチャが有効である。
また、キーワードパッケージにインターンされるシンボル（:から始まるシンボル）は、定数として自己評価される特性を持つ[21]。これらのシンボルはロード時にインスタンス化され、valueスロットには自身への参照が初期値として格納されるようコンパイラが手配する。

クロージャ (Closure) と関数参照表現

Common Lispにおいて、関数はファーストクラスのオブジェクトであり、レキシカルスコープの環境を捕捉したクロージャとして振る舞う。Wasm自体はレキシカルクロージャをネイティブにサポートしていないため、クロージャ表現への変換（Closure Conversion）をコンパイラフェーズで実施し、コードポインタと捕捉された環境（自由変数）をペアにした構造体としてエミットする必要がある[23]。
クロージャのWasmGC表現は、関数ポインタと環境ベクタの2つのフィールドを持つ構造体となる。関数ポインタは、実行されるWasm関数の参照であり、WasmGCにおける(ref $func_signature)型を利用し、ref.func命令で取得される[25]。環境は、捕捉されたレキシカル変数を格納するWasmGC配列、または専用に生成された固定長の構造体として表現される。

クロージャの呼び出し時には、まず構造体から関数参照（funcref）を取り出し、次にcall_ref命令（または末尾呼び出しの場合はreturn_call_ref命令）を用いてディスパッチを行う[25]。このアプローチは、Wasmの関数テーブルとインデックス（call_indirect）を用いる従来の手法と比較して、単一のディスパッチテーブルによる局所性の向上や、不要なインダイレクションの排除による実行速度の改善をもたらす[26]。さらに、ホストのJITコンパイラはcall_refの対象が不変であることを検知しやすく、積極的なインライン化が可能となる。

実行時スタックと呼出規約 (Calling Conventions)

WebAssemblyの関数呼び出しは強力に静的型付けされており、定義されたシグネチャと正確に一致する数の引数を提供しなければコンパイルエラーとなる[27]。対照的に、Common Lispの関数は極めて柔軟なラムダリスト（&optional、&rest、&key、&allow-other-keys）をサポートしており、関数呼び出し時に提供される引数の数は動的に変動する[29]。このインピーダンスミスマッチを解消するためには、呼出規約（Calling Convention）をコンパイラレベルで仮想化する必要がある[14]。

ラムダリストと引数渡しの仮想化戦略

コンパイラが採用しうるラムダリストの翻訳戦略として、いくつかのアプローチが存在する。これらの戦略はパフォーマンスとコンパイル時の複雑さに直接的な影響を与える。

戦略	アーキテクチャと**Wasm**表現	評価（メリットとデメリット）
引数配列化 **(Array-based)**	すべてのLisp関数を (func (param (ref $array)) (result anyref)) の単一シグネチャに統一する。呼び出し側はすべての引数を動的に配列に詰めて渡し、受信側が配列を解析する。	長所: 動的ディスパッチ（applyなど）の処理が極めて単純化される。[br] 短所: 呼び出しのたびにヒープ割り当てが発生し、GC圧力が劇的に増大する。パフォーマンス上の致命的なボトルネックとなり得る。
ハイブリッド渡し **(Register & Stack)**	必須引数（Required）はWasmのパラメータとして渡し、可変引数（&rest、&key）は呼び出し側が追加のスタックまたはバッファ配列に格納する（C言語やGoの呼出規約に類似）[32]。	長所: 必須引数のみの関数の場合、Wasmエンジンのレジスタ割り当てが最適に機能する。[br] 短所: コンパイラのコード生成ロジックが複雑化し、実行時のシグネチャミスマッチを管理する負担が増える。
明示的スタック仮想化 **(Explicit Stacks)**	Guile Hootが採用する手法。関数をCPS変換し、引数やラ	長所: 継続のキャプチャが容易になり、動的arityの問題を

	イブ変数を明示的なスタック（線形メモリや参照配列）にプッシュしてからtail_callを行う[14]。	完全にバイパスできる。[br]短所: 単純な関数呼び出しにもスタック操作のオーバーヘッドが乗るため、高度な最適化パス（インライン化や不要なスタック操作の除去）が必須となる。

Common Lispの場合、必須引数のみで構成される関数呼び出しが圧倒的多数を占めるため、基本的には必須引数をWasmのネイティブパラメータとして渡しつつ、可変長部分（&restや&key）が発生しうる部分にのみ「引数スライス」としてWasmGC配列を付加するハイブリッドアプローチが現実的である。特にキーワード引数（&key）の解析において、呼び出しのたびにリストを辿って探索するのは計算コストが高い。コンパイル時に与えられたキーワード群が静的に決定できる場合、呼び出し側でそれらを配列の固定インデックスに再配置（パーフェクトハッシュ化）し、受信側関数では直接array.getで取り出せるようにする最適化が不可欠である。

多値 (Multiple Values) の処理と限界

Common Lispの強力な機能の一つに、valuesおよびmultiple-value-bindを用いた多値の返却がある[24]。WebAssemblyの多値提案（Multi-Value Proposal）は、関数が複数の値をスタックに残してリターンすることを可能にしており（例: [t*] -> [t*]）[38]、一見するとこの機能が直接利用できるように思われる。

しかし、Lispの多値とWasmの多値には根本的な意味論の不一致がある。Wasmでは、返される値の数と型は関数のシグネチャとして厳格に固定される。一方で、Common Lispでは、関数の呼び出し側は返された多値を無視して単一の値として受け取ることも、期待した数より少ない値が返された場合にNILで埋めることも許容されている。すなわち、Lispの多値は動的（Dynamic arity）であり、呼び出し側と受信側の間で返される値の数がコンパイル時に一致している保証はない。

この矛盾を解決するため、Lispにおける多値は、Wasmのネイティブな複数値リターンに直接マッピングするのではなく、実行環境（スレッドローカルな状態）内に「多値リターン用バッファ（Multi-value return buffer）」を設ける手法が推奨される。関数は常に第一返り値をWasmの標準リターン値として返し、追加の値が存在する場合のみ、その大域バッファに書き込む。multiple-value-bindは、第一値を受け取った後、付随する値の有無をバッファ長から確認し、必要な数の値を取り出すというメカニズムを構築する。この設計により、Wasmの厳格な型検証を通過しつつ、Common Lispの柔軟な多値セマンティクスを完全に再現できる。

動的スコープとスペシャル変数 (Dynamic Scope)

Common Lispはレキシカルスコープを基本としつつも、defvarやdefparameterによって宣言されたスペシャル変数を通じた動的スコープ（Dynamic Binding）をサポートしている[40]。レキシカル変数がWasmのローカル変数（local.get / local.set）やクロージャの環境配列にマッピングされるのに対し[43]、動的変数は実行コンテキスト全体で値を共有し、シャドウイングの履歴をスタックとして保持しなければならない[44]。

C言語への変換をベースとした歴史的なLisp実装（KCLなど）では、シャローバインディング（Shallow Binding）と呼ばれる手法が採用されてきた[22]。このアプローチはWasmGCの設計にも非常に適して

いる。シンボル構造体が持つvalueスロットには常に現在の動的バインディング値が格納される。関数の内部でスペシャル変数がlet等によってシャドウイング（動的束縛）される際、コンパイラはシンボルの現在のvalueスロットの値を、大域的な「バインディングスタック（Binding Stack）」にプッシュし、その変数の参照と共に記録するシーケンスを生成する[22]。そして、新しい値をシンボルのvalueスロットに書き込む。変数のスコープを抜ける際（通常の終了、または例外による大域脱出を含む）、バインディングスタックから以前の値をポップし、対象シンボルのvalueスロットを復元する[22]。
Wasmにおけるバインディングスタックは、線形メモリ上に構築するか、あるいはWasmGCの大域配列として管理することができる。検索コストがゼロであるため、グローバルな状態へのアクセス頻度が高いCommon Lispプログラムにおいて、シャローバインディングはディープバインディング（環境リストの探索）よりも圧倒的なパフォーマンス優位性をもたらす。

大域脱出と例外処理 (Exception Handling / Non-local Exits)

Common Lispは、強力かつ柔軟な制御構造として大域脱出（Non-local exits）を提供する。代表的なものに、TAGBODYとGO（ローカルなジャンプ）、BLOCKとRETURN-FROM（レキシカルな大域脱出）、そしてCATCHとTHROW（動的な大域脱出）がある。これらの機構は、Wasmの制御フロー命令と例外処理提案（Exception Handling Proposal: Wasm EH）を組み合わせて表現される。

ローカル制御フロー: BLOCK, TAGBODY

Wasmは構造化された制御フロー（Structured Control Flow）を採用しており、任意の場所へのgoto文を持たず、ネストされたblockやloopに対する分岐命令（brやbr_if）のみを許可している[46]。
Common LispのBLOCKは、Wasmのblock命令に直接マッピングされる。指定されたブロック名に基づくRETURN-FROMは、対応するブロックレベルへの前方ジャンプ命令（br）としてコンパイルされ、返り値をWasmスタックに積むことで実現される[46]。また、TAGBODY内部のタグ付けされたラベル群は、ネストされた一連のWasm blockとloopに変換される。GO命令は、これらのラベルに対応するブロックの深さを計算してbr命令を発行する。これらは関数内での静的に解決可能なジャンプであるため、実行時のオーバーヘッドは生じない。

動的脱出とWasm例外処理 (Wasm EH)

問題となるのは、呼び出し元関数のスコープを飛び越えて動的に脱出するCATCHとTHROW、およびコンディションシステムの実装である。これらは関数境界を越えるスタックの巻き戻し（Unwinding）を伴い、同時にUNWIND-PROTECT（クリーンアップ処理の保証）を確実に実行しなければならない。過去のWasm環境では、これを実現するためにShadow Stack（独自のスタック管理）やJavaScript例外へのフォールバックを利用していたが、現在のWasmにはネイティブな例外処理（EH）機構が備わっている[48]。最新のWasm EH提案では、字句スコープに依存した古いtry/catch命令は廃止され、より堅牢なラベルベースの制御フローであるtry_table、throw、throw_ref、および各種catch節（catch、catch_ref、catch_all、catch_all_ref）が導入されている[49]。
Common LispのCATCH/THROWをWasm EHで実装するアーキテクチャは、以下のように構成される。

1. タグの定義: Wasmのモジュール内で、汎用的なLisp例外を示す単一の例外タグ（例: $lisp_exception）を定義する。このタグは、スローされるデータのペイロードとして「Lispのタグ

オブジェクト（シンボル等）」と「渡される値」の2つを受け取るよう定義する[53]。

2. **THROW**の変換: Lispの(throw 'my-tag value)は、タグオブジェクトと値をスタックに積み、Wasmのthrow $lisp_exception命令を発行するコードにコンパイルされる[49]。
3. **CATCH**と**try_table**: Lispの(catch 'my-tag ...)は、Wasmのtry_tableブロックにラップされる[49]。
   コード スニペット
   (block $catch_block (result anyref exnref)
    (try_table (catch_ref $lisp_exception $catch_block)
      ;; ... catch内部の評価コード ...
)
)

4. タグの動的検証と再スロー: try_tableによって捕捉された例外オブジェクト（exnref）は、Lispレベルでのタグオブジェクト（例: 'my-tag）が一致するかどうかを実行時に検査されなければならない[55]。Wasmのタグは静的（IDベース）であるのに対し、Lispのタグは動的（値ベース）であるという本質的な相違があるためである。検査の結果、タグが一致すればそのペイロードを返り値とし、一致しなければ、throw_refを用いて捕獲した例外（exnref）をそのまま上位のフレームへ再スローする[49]。

このアーキテクチャにより、スタックの巻き戻し処理はホストVMに完全に委譲される。さらにUNWIND-PROTECTにおけるクリーンアップも、例外をcatch_all_refで一旦キャッチしてクリーンアップ処理を実行した後に、throw_refで元の脱出プロセスを再開するというパターンで効率的に記述可能となる[52]。このプロセスの中で、前述のバインディングスタック（スペシャル変数の復元）の巻き戻し処理も正確にインターリーブさせる必要がある。

末尾呼び出し最適化と継続

関数型プログラミングにおいて、末尾呼び出し最適化（Tail Call Optimization: TCO）は単なる最適化ではなく、再帰による無限ループを安全に記述するための必須のセマンティクスである。WebAssemblyのTail Call提案はすでに標準化されており、return_callおよび（WasmGC向けの）return_call_ref命令を提供している[36]。コンパイラは、関数の末尾位置にある呼び出しをこれらの命令に変換することで、Wasmコールスタックの消費を防ぐことができる。

より高度な制御構造である「限定継続（Delimited Continuations）」を実装する場合、ネイティブのWasmスタックだけでは不十分である。Guile Hootプロジェクトにおける洞察が示すように、Wasm上で継続を第一級オブジェクトとして切り出すためには、プログラム全体を継続渡しスタイル（CPS: Continuation-Passing Style）に変換し、すべての関数呼び出しを非同期的な状態遷移としてモデル化するか、あるいはコールスタック自体を線形メモリ上に明示的にアロケーションして仮想化するアプローチが必要となる[14]。ただし、Common Lispの標準仕様はファーストクラスの継続（Schemeのcall/cc）を要求していないため、極端なCPS変換は不要である可能性が高い。標準的なTCOにはreturn_call_refを使用し、大域脱出には前述の例外処理メカニズムを利用することで、Wasmエンジンのネイティブな実行速度を最大限に引き出す設計が望ましい。

メモリ管理とガベージコレクション (GC) の統合

WasmGCの最大の恩恵は、コンパイラ開発者が複雑でバグを生みやすいマーク・アンド・スイープや

世代別ガベージコレクタを自ら記述する義務から解放されることである。WasmGCを用いて割り当てられた構造体（struct.new）や配列（array.new）は、ホスト環境（ブラウザであればV8やSpiderMonkeyなど）のGCヒープ上に直接配置される[7]。メモリが逼迫した場合、array.newなどのアロケーション命令は同期的にガベージコレクションをトリガーし、不要なヒープ領域を自動的に解放する[19]。
このホスト統合がもたらす技術的影響として、パフォーマンスの大幅な向上が挙げられる。ホストVMは、メモリプレッシャーを監視し、ヒープサイズやGC頻度を動的に調整する高度なヒューリスティクスを備えている[7]。Common LispのオブジェクトがWasmGCを通じて管理されることで、ブラウザ環境ではLispオブジェクトとJavaScriptオブジェクトが同一のメモリ領域、同一のコレクタサイクルのもとで処理される。これにより、両言語間でオブジェクトを受け渡す際のシリアライズ/デシリアライズや、メモリの二重管理といった深刻なオーバーヘッドが消滅し、真のゼロコピー（Zero-copy）相互運用が達成される[3]。
また、Wasmには、ホスト環境（JS等）の不透明なオブジェクトを保持するためのexternrefという型が存在する[12]。WasmGCのanyref階層とexternref階層は厳密に分離されており、両者が直接交わることはない[12]。この設計は、JSエンジン側が自らの整数表現とWasmのi31refとを区別するコストを排除するためのものである[12]。Common Lispの環境からDOMオブジェクトなどのJavaScriptネイティブオブジェクトを操作する場合、Lispのオブジェクトシステム（コンスや構造体）内にexternrefを直接格納することは型システムの制約上できない。そのため、Lisp側で外部オブジェクトを管理するための専用のラッパー構造体（struct内部にexternrefへの変換テーブルインデックスを持つなど）を介して安全なバウンダリを構築し、キャストの安全性を担保する設計が必要となる。

結論

本仕様書が明らかにしたように、WebAssembly GCおよびException Handling（例外処理提案）の登場は、Common Lispのような高度に動的でリッチなセマンティクスを持つ言語を、ブラウザやクラウドネイティブなWasmランタイム上で高速に動作させるためのパズルの最後のピースである。
コンパイラ開発において決定的に重要な点は、過去のC言語トランスパイラや線形メモリ依存の設計パラダイムを捨て去り、Wasmネイティブな構造に完全に適応することである。具体的には、データ型の表現において汎用データ型として(ref null eq)を基底とし、ヒープ割り当てを回避できるi31refをFixnumや文字に最大限活用するアプローチが求められる[10]。さらに、制御フローの領域では、Lispの動的脱出（CATCH/THROW）やUNWIND-PROTECTを、新しいラベルベースのtry_tableとthrow_refに厳密にマッピングすることで、スタック操作の安全性をVMに委譲する[49]。そして呼出規約においては、Wasmの静的な関数シグネチャとLispの柔軟なラムダリストとの間にある意味論のギャップを埋めるため、多値リターンバッファや引数スライスを併用したスマートなハイブリッド呼出規約を実装する必要がある[27]。
これらの技術仕様に基づくコンパイラ実装は、ランタイムの肥大化を防ぎ（KB単位のバイナリサイズを実現[4]）、V8などの強力なJITおよびGC最適化を完全に享受できる[7]。WasmGCをターゲットとするCommon Lisp処理系は、単なる実験的プロトタイプを超え、エンタープライズ規模のウェブアプリケーション開発や、エッジコンピューティング環境における実用的な選択肢として、強力なパフォーマンスと表現力を提供する基盤となるはずである。

引用文献

1. Introduction (Guile Hoot) - Spritely Institute, https://files.spritely.institute/docs/guile-hoot/0.1.0/Introduction.html
2. WebAssembly - Wikipedia, https://en.wikipedia.org/wiki/WebAssembly
3. WebAssembly Garbage Collection（WasmGC） - Midori Photo Art, https://yosuke4061.com/new_toppage/powerword/webassembly-garbage-collection/
4. Scheme in the browser: A Hoot of a tale - Spritely Institute, https://spritely.institute/news/scheme-wireworld-in-browser.html
5. google/schism: A self-hosting Scheme to WebAssembly compiler, https://github.com/google/schism
6. Introduction (Guile Hoot) - Spritely Institute, https://files.spritely.institute/docs/guile-hoot/0.8.0/Introduction.html
7. A new way to bring garbage collected programming languages, https://v8.dev/blog/wasm-gc-porting
8. gc/proposals/gc/Overview.md at main · WebAssembly/gc - GitHub, https://github.com/WebAssembly/gc/blob/main/proposals/gc/Overview.md
9. Scheme in Scheme on Wasm in the browser - Spritely Institute, https://spritely.institute/news/scheme-in-scheme-on-wasm-in-the-browser.html
10. wasm gc proposal 勉強 - Heap Types - Zenn, https://zenn.dev/tanishiking/scraps/7347e2298524cb
11. gc/proposals/gc/MVP.md at main - WebAssembly - GitHub, https://github.com/WebAssembly/gc/blob/main/proposals/gc/MVP.md
12. i31 as a ref type dimension like null · Issue #130 · WebAssembly/gc, https://github.com/WebAssembly/gc/issues/130
13. WasmGCで導入される型や命令のお勉強 - Zenn, https://zenn.dev/tanishiking/articles/learn-wasm-gc
14. ABI.md - main · spritely / Guile Hoot - GitLab, https://gitlab.com/spritely/guile-hoot/-/blob/main/design/ABI.md
15. cast: Wasm reference instruction - WebAssembly - MDN Web Docs, https://developer.mozilla.org/en-US/docs/WebAssembly/Reference/Reference/cast
16. Why do we need the type of i31 in WasmGC proposal?, https://stackoverflow.com/questions/77468063/why-do-we-need-the-type-of-i31-in-wasmgc-proposal
17. Compiling Scheme to WebAssembly - Eli Bendersky's website, https://eli.thegreenplace.net/2026/compiling-scheme-to-webassembly/
18. WebAssembly Core Specification - W3C, https://www.w3.org/TR/wasm-core-2/
19. ArrayRef in wasmtime - Rust, https://docs.wasmtime.dev/api/wasmtime/struct.ArrayRef.html
20. The less-familiar parts of Lisp for beginners — packages and symbols, https://blog.cneufeld.ca/2014/02/the-less-familiar-parts-of-lisp-for-beginners-packages-and-symbols/
21. 21. Programming in the Large: Packages and Symbols - gigamonkeys, https://gigamonkeys.com/book/programming-in-the-large-packages-and-symbols

22. Design and Implementation of Kyoto Common Lisp, https://softwarepreservation.computerhistory.org/LISP/kcl/paper/kcl-paper.pdf
23. cps in hoot - wingolog, https://wingolog.org/archives/2024/05/27/cps-in-hoot
24. approaching cps soup - wingolog, https://wingolog.org/archives/2023/05/20/approaching-cps-soup
25. Typed Function References for WebAssembly - proposals - GitHub, https://github.com/WebAssembly/gc/blob/main/proposals/function-references/Overview.md
26. Understanding Overhead · Issue #249 · WebAssembly/gc - GitHub, https://github.com/WebAssembly/gc/issues/249
27. a world to win: webassembly for the rest of us - wingolog, https://www.wingolog.org/archives/2023/03/20/a-world-to-win-webassembly-for-the-rest-of-us
28. Understanding WebAssembly text format - MDN Web Docs, https://developer.mozilla.org/en-US/docs/WebAssembly/Guides/Understanding_the_text_format
29. Common lisp: Can I define a function with arbitrary number of args, https://stackoverflow.com/questions/22805867/common-lisp-can-i-define-a-function-with-arbitrary-number-of-args-and-optional
30. Functions - gigamonkeys, https://gigamonkeys.com/book/functions
31. Chapter 21 - Fancy Tricks with Function and Macro Arguments, https://dept-info.labri.fr/~strandh/Teaching/PFS/Common/David-Lamkins/chapter21.html
32. x86 calling conventions - Wikipedia, https://en.wikipedia.org/wiki/X86_calling_conventions
33. Calling convention - TinyGo, https://tinygo.org/docs/concepts/compiler-internals/calling-convention/
34. tool-conventions/BasicCABI.md at main - WebAssembly - GitHub, https://github.com/WebAssembly/tool-conventions/blob/main/BasicCABI.md
35. Calling Conventions in C/C++ - GeeksforGeeks, https://www.geeksforgeeks.org/cpp/calling-conventions-in-c-cpp/
36. WebAssembly for the rest of us 17 Mar 2023 - wingolog, https://wingolog.org/pub/2023-bobkonf-wasm-for-the-rest-of-us-slides.pdf
37. Multi-Value All the Wasm - Hacker News, https://news.ycombinator.com/item?id=21596965
38. Multi-Value All The Wasm! - Bytecode Alliance, https://bytecodealliance.org/articles/multi-value-all-the-wasm
39. Multi-Value All The Wasm! - Mozilla Hacks - the Web developer blog, https://hacks.mozilla.org/2019/11/multi-value-all-the-wasm/
40. the binding concept on LISP programming language - Stack Overflow, https://stackoverflow.com/questions/40816760/the-binding-concept-on-lisp-programming-language
41. Dynamic Binding and Macros - common lisp - Stack Overflow, https://stackoverflow.com/questions/44600212/dynamic-binding-and-macros
42. Notes on Refactoring Exponential Macros in Common Lisp Or - arXiv,

https://arxiv.org/pdf/2305.02991
43. Manipulating the Wasm Stack Pointer from JavaScript (for Clang), https://zenn.dev/herumi/articles/wasm-stack-operation?locale=en
44. Dynamic Binding in Common Lisp - Stack Overflow, https://stackoverflow.com/questions/64418124/dynamic-binding-in-common-lisp
45. Lisp compilers that compile directly to assembly - Chris Done, https://chrisdone.com/posts/lisp-compilers/
46. Towards a WasmGC backend and runtime for the MicroHs compiler, https://studenttheses.uu.nl/bitstreams/fd20cc05-b43b-4b61-880a-ff93065d7e45/download
47. WebAssembly Specification, https://webassembly.github.io/exception-handling/core/_download/WebAssembly.pdf
48. C++ Exceptions Support - Emscripten 6.0.14-git (dev) documentation, https://emscripten.org/docs/porting/exceptions.html
49. Exceptions in Cranelift and Wasmtime - Bytecode Alliance, https://bytecodealliance.org/articles/wasmtime-exceptions
50. Implement the new wasm Exception Handling proposal [42204334], https://issues.chromium.org/issues/42204334
51. Adding Support for Exception Handling · Issue #1884 - GitHub, https://github.com/wasm-micro-runtime/wasm-micro-runtime/issues/1884
52. try_table: Wasm exception handling instruction - WebAssembly | MDN, https://developer.mozilla.org/en-US/docs/WebAssembly/Reference/Exception_handling/try_table
53. throw: Wasm exception handling instruction - MDN Web Docs - Mozilla, https://developer.mozilla.org/en-US/docs/WebAssembly/Reference/Exception_handling/throw
54. throw_ref: Wasm exception handling instruction - WebAssembly | MDN, https://developer.mozilla.org/en-US/docs/WebAssembly/Reference/Exception_handling/throw_ref
55. nominal types in webassembly - wingolog, https://wingolog.org/archives/2026/03/10/nominal-types-in-webassembly
56. Exception Handling in GraalWasm - System Software, https://ssw.jku.at/Teaching/BachelorTheses/2025/Windischbauer_Valentin.pdf
57. Conventions — WebAssembly 2.0 + tail calls (Draft 2023-03-01), https://webassembly.github.io/tail-call/core/exec/conventions.html